SimpleStoreServiceOrchestration

Interface Design Description

Eclipse Arrowhead 5.2.2 | core system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple. 11 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-simple.

SimpleStoreServiceOrchestration - Go 5.2.2

Standalone core system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/SimpleStoreServiceOrchestration. Entry point: ./cmd/serviceorchestration-simple. Module: arrowhead/ah5-core-go/serviceorchestration-simple. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. Existing role selection and API behavior are retained. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8442 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8442". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../../LISTENING-PORTS.md) for precedence and all systems.

Detailed implementation evidence

0 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Go HTTP route inventory

This is the repository route inventory, not a new differential test. Route presence does not establish DTO, error, authorization or persistence equivalence.

- GET /serviceorchestration/general/mgmt/get-config | Go route recorded: True

- POST /serviceorchestration/general/mgmt/logs | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/history/query | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/push/query | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/push/subscribe | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/push/trigger | Go route recorded: True

- DELETE /serviceorchestration/orchestration/mgmt/push/unsubscribe | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/simple-store/create | Go route recorded: True

- PUT /serviceorchestration/orchestration/mgmt/simple-store/modify-priorities | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/simple-store/query | Go route recorded: True

- DELETE /serviceorchestration/orchestration/mgmt/simple-store/remove | Go route recorded: True

- POST /serviceorchestration/orchestration/pull | Go route recorded: True

- POST /serviceorchestration/orchestration/subscribe | Go route recorded: True

- DELETE /serviceorchestration/orchestration/unsubscribe/{id} | Go route recorded: True

Native compatibility profile

Compatibility profile and remaining work

This document is deliberately separate from the route inventory. It records the actual implemented behavior and material gaps, rather than calling a set of HTTP handlers a complete Java replacement.

Implemented

- Authentication: PASSWORD identities, bcrypt cost 10, batch management, credential changes, credential-authenticated logout, UUID sessions, one active session per identity, expiration checks, session query/closure and identity query filters. Credentials and session tokens are excluded from management responses. Credential changes and management updates invalidate sessions (a stricter security choice).

- ConsumerAuthorization: provider and MGMT policy levels, MGMT precedence, ALL/WHITELIST/BLACKLIST/SYS_METADATA policies, scoped/default fallback, all-scopes checks, grant/lookup/query/revoke/check operations. Missing policies deny access.

- Authorization tokens: usage/time-limited tokens; URL-base64 self-contained tokens with the Java pipe-separated field order; RS256/RS512 JWTs with psn/csn/ccn/tat/tan/sco claims; public-key export; management generation/query/revocation; provider-bound online verification with atomic usage decrement. Raw tokens are returned only at generation, not retained for later retrieval.

- Encryption-key registration: Java AES/ECB/PKCS5Padding and AES/CBC/PKCS5Padding wire encodings, provider-specific IVs for CBC, owner/management removal. Provider keys are authenticated-encrypted at rest with AES-GCM and provider identity as additional authenticated data. The server certificate's RSA key signs JWTs.

- Simple orchestration: consumer-specific store, unique priorities per consumer/service, priority modification, preferred providers, ONLY_PREFERRED, per-service MATCHMAKING, ignored-field/flag warnings, pull/history.

- Dynamic orchestration: service discovery through ServiceRegistry, mandatory authorization checks, preferred providers, interface operations, token acquisition, MATCHMAKING, provider-consented exclusivity capped by allowExclusivity, atomic lock conflict rechecks, lock management, pull/history.

- Both orchestrators: persisted subscriptions, owner checks, management subscription/query/unsubscribe/trigger operations, persisted jobs and HTTPS push callbacks. Callback TLS is pinned to the target system identity before sending results. Interrupted in-progress jobs become ERROR on restart instead of being redelivered automatically.

- General management: sanitized configuration map, bounded persisted audit log querying, authenticated echo. Source-generated service descriptions and opt-in periodic service registration.

- Shared runtime: mandatory local-cloud mTLS, identity-token certificate binding, fixed authority identity pinning before credentials, redirect rejection, coalesced login refresh, conditional token invalidation, body limits, server timeouts, atomic JSON snapshots and exclusive process ownership of a state file.

- ServiceRegistry: pre-existing Go implementation plus corrected AuthorizationVerifyRequestDTO fields/operation names and removal of an implicit Sysop-name privilege escalation in outsourced token verification. Its original configuration and publisher are preserved.

Not implemented / not equivalent

1. MQTT APIs (since 2026-10-24) differ from Java in three deliberate points: the response topic must lie under arrowhead/response/<requester>/, requests with an invalid identity token are not answered (no 401 response), and identity login/logout/change stay HTTPS only. Request topics, envelopes, operation names and error payloads follow the Java MQTT API; see "Changes 2026-10-24 (MQTT)". 2. Dynamic federation, translation and QoS integration are absent. Requests that enable ALLOW_INTERCLOUD, ONLY_INTERCLOUD or ALLOW_TRANSLATION, or supply QoS requirements, fail explicitly with 501. They are not silently treated as successful local equivalents. Foreign-consumer management token generation also returns 501. This is unsuitable for a federated orchestration replacement until those paths are ported and tested. 3. Blacklist support and authorization-driven management filters are not ported to the four new systems. Management uses identity sysop privileges or an explicit whole-management whitelist. This is less granular than per-operation authorization; do not use the whitelist as a substitute for fine-grained policy enforcement. 4. The new systems do not yet publish native sysMonitor/AMQP telemetry. Their echo APIs and history/logs exist, but they will not satisfy a LocalCloudManager native-telemetry readiness requirement. The existing ServiceRegistry publisher remains available. 5. No Java SQL database compatibility/import. Each new system uses a private atomic JSON state file, not JPA/MySQL. Never mount a Java database volume as a Go state file. Java identity, authorization, key, store, lock, subscription and history data need a separately reviewed importer before migration. 6. Configuration is JSON, not Spring properties, environment override semantics, keystores or Java's full property catalog. get-config returns a sanitized native configuration subset in the Java map response wrapper. There is no cloud registration health/readiness gate; registration failures are retried and reported to the process log. 7. Validation/normalization and query semantics are not exhaustively matched. Supply canonical names and enum spelling. Field lengths, case normalization, HTTP error details/statuses, empty/null distinctions, ordering/default page sizes and warning text still need Java golden-contract tests. Pagination is bounded, but is not a full port of Java's per-entity paging service. JSON inputs use dynamic documents rather than generated, exhaustive DTO validation. 8. Metadata supports the documented comparisons and dotted/indexed paths, but REGEXP uses Go RE2, not Java Pattern; lookarounds/backreferences are not supported. Mixed-type numeric/collection edge cases need further Java fixture tests. 9. Dynamic candidate ranking is deterministic local/preferred ordering, not the complete Java matchmaking/QoS/fallback pipeline. Scopes are checked fail-closed; a denied requested operation can reject orchestration instead of yielding Java's precise partial-result behavior. Token acquisition is not rolled back if a later lock conflict aborts orchestration. 10. Push uses one worker and one delivery attempt, without Java's Quartz/thread-pool behavior, HTTP/plain MQTT callbacks, retries, or automatic event-driven re-orchestration. HTTPS result-delivery behavior is tested; duplicate delivery avoidance does not constitute exactly-once delivery. Notifications interrupted by a crash are reported as uncertain ERROR. 11. Logs are native bounded audit records (1,000 retained), not the full Java logging database or JVM-specific exceptions. Sessions/tokens/jobs/subscriptions/locks are validated for expiry at access but do not have complete Java cleaner/retention policies; state can grow over time. Expired entities may differ in query visibility. 12. No certificate revocation enforcement beyond Go's ordinary certificate-chain/time/hostname checks. No blacklist, rate limiter, distributed transactions, HA, distributed locks, SQL migration, or production security certification. Offline self-contained token revocation requires provider-side expiration/revocation handling; online revocation does not invalidate already-issued offline tokens.

Release gate

Do not describe this profile as fully interchangeable with the Java cloud. Before replacement: implement the required missing capabilities, compare successful and failing requests against Java for every DTO/operation, verify all discovery/security policies, migrate data with rollback, verify monitoring/UI readiness and federation, and rerun the cloud security evaluation. None of those deployment changes is made by this source delivery.

Changes 2026-10-17 (GoCloud deployment)

- Dynamic orchestration passes each requested operation as an authorization scope; a provider qualifies only if

every requested operation is granted. Without operations the all-scopes check applies (as before). Test: TestNativeServiceRegistryIntegration.

- Deployed in Arrowhead-test-cloud-go (see its GO-NATIVE-FULL-2026-10-17.md) by operator decision, with the gaps

listed above accepted; the release gate above is not thereby satisfied.

- Inter-cloud orchestration: ALLOW_INTERCLOUD (fallback when no local provider) and ONLY_INTERCLOUD through the local

Gatekeeper (gatekeeperURL): globalServiceDiscovery query, interCloudNegotiation negotiate, GateTunnel interface result. ALLOW_TRANSLATION and QoS requirements still return 501.

- Native sysMonitor telemetry (monitorEnabled, amqpURL): sysMonitor offer on generic_amqps, echo/power/status.

- managementPolicy: authorization: ConsumerAuthorization decides management access per service and operation.

- Requested operations match string-set operations (MQTT/AMQP templates) as well as HTTP operation maps.

- Push subscriptions store notifyRecipient: a non-sysop management subscriber (orchestration by proxy) receives

the result; own and sysop subscriptions deliver to the target.

Changes 2026-10-24 (translation)

- ALLOW_TRANSLATION: an interface is translatable when it does not match the template or property requirements

natively (as Java assortInterfacesAndMarkIfNonNative), so a bridge can also translate only the data model of the accepted template (generic_https senml -> generic_https simpleJson). Native candidates still win; the consumer's authorization for the provider is still checked before a bridge is built.

Changes 2026-10-24 (MQTT)

- MQTT API of all five core systems (internal/core/mqtt.go, ServiceRegistry/mqtt.go), the Java mqtt.api.enabled

counterpart: the Java request envelope (MqttRequestTemplate: traceId, authentication IDENTITY-TOKEN//<token>, responseTopic, qosRequirement, params, payload) on the Java base topics + operation (arrowhead/serviceregistry/{device-discovery,system-discovery,service-discovery,management,general/management, monitor}/, arrowhead/authentication/identity/[management/], arrowhead/consumer-authorization/authorization[-token]/ [management/], arrowhead/serviceorchestration/orchestration/[management/{push,history,lock,store,plant-rules}/], general/management/, monitor/echo); the response (MqttResponseTemplate: status, traceId, receiver, payload; errors as ErrorMessageDTO with exceptionType and origin = topic) goes to the request's response topic.

- The adapter verifies the identity token and runs the request in process through the HTTPS route, so the blacklist,

management (whitelist/authorization) checks and handlers are the same as over HTTPS. Path parameters come from params (a single one also from a string payload, as Java), the other params become the query, a list payload of a removal operation is its list parameter (names, instanceIds, uuids, ids, tokenReferences, systemNames, serviceInstances, keys).

- Response binding (Go): the response topic must lie under arrowhead/response/<system of the token>/; with a broker

ACL that lets each client read only its own prefix, a stolen token cannot be used to read answers.

- Configuration: mqttURL (ssl://host:port, client certificate = system certificate) and mqttTopicPrefix (a second

orchestrator on the same broker needs another prefix); ServiceRegistry mqtt.api.enabled, mqtt.broker.address, mqtt.broker.port, mqtt.topic.prefix (requires TLS and the outsourced policy). With MQTT enabled every service is also registered with a generic_mqtts interface (baseTopic, operation list), as Java.

- ServiceRegistry: the Blacklist filter never checks Blacklist itself (Java BlacklistFilter); a Java Blacklist, which

registers before it serves its check, was refused under force.blacklist.filter.

- cmd/mqtt-call: command-line client for one MQTT API request (tests).

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-core-rust/serviceorchestration-simple.

SimpleStoreServiceOrchestration (Rust)

Arrowhead 5.2.2 core system. src/main.rs is built as binary serviceorchestration-simple by the arrowhead-core-rust crate in ... serviceorchestration-simple.json and documentation/ link to ../examples and ../documentation/SimpleStoreServiceOrchestration.

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-generalManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-generalManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-sysMonitor-AMQP-IDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-core-rust/documentation/SimpleStoreServiceOrchestration/SimpleStoreServiceOrchestration-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

generalManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct KeyValues . . . 6
3.3 struct LogEntry . . . 6
3.4 struct LogEntryListResponse . . . 6
3.5 struct LogRequest . . . 6
3.6 struct Page . . . 7
3.7 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `generalManagement` service interface of the SimpleStoreServiceOrchestration system, which provides general management of the SimpleStoreServiceOrchestration system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `generalManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8442 and base path `/serviceorchestration/general/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`get-log`	POST	`/logs`
`get-config`	GET	`/get-config`

2 Service Operations

The interfaces of the `generalManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `generalManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/general/mgmt/logs

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/general/mgmt/logs HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 1: An get-log invocation.

```
{
  "entries": [
    {
      "logId": "42",
      "entryDate": "2026-10-22T12:00:00Z",
      "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService",
      "severity": "INFO",
      "message": "Service instance registered: TemperatureSensor|temperatureReading|1.0.0"
    }
  ],
  "count": 1
}
```

Listing 2: An get-log response.

2.2 GET /serviceorchestration/general/mgmt/get-config?keys=*{keys}*

Operation: get-config
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

Query parameters: *keys* (List<String>, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /serviceorchestration/general/mgmt/get-config?keys=<keys> HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-config invocation.

```
{
  "map": {
    "server.port": "8443"
  }
}
```

Listing 4: An get-config response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.3 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.4 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.6 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-generalManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 5

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct KeyValues . . . 6
3.5 struct LogEntry . . . 6
3.6 struct LogEntryListResponse . . . 7
3.7 struct LogRequest . . . 7
3.8 struct Page . . . 7
3.9 Primitives . . . 7

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `generalManagement` service interface of the SimpleStoreServiceOrchestration system over MQTT, which provides general management of the SimpleStoreServiceOrchestration system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/general/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`get-log`	`arrowhead/serviceorchestration/general/management/get-log`
`get-config`	`arrowhead/serviceorchestration/general/management/get-config`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/general/management/get-log`

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 3: Payload of an get-log request.

2.2 PUBLISH `arrowhead/serviceorchestration/general/management/get-config`

Operation: get-config
Input: Array<String>
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

```
[
  "string"
]
```

Listing 4: Payload of an get-config request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.5 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.6 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.7 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.8 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-generalManagement-SD.pdf.

[6] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-generalManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestration HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestration"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestration` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation pull . . . 4
2.2 operation subscribe . . . 5
2.3 operation unsubscribe . . . 7

3 Data Models **8**
3.1 struct AuthorizationTokenGenerationResponse . . . 8
3.2 struct ErrorMessage . . . 8
3.3 struct MetadataRequirement . . . 8
3.4 struct OrchestrationNotifyInterface . . . 8
3.5 struct OrchestrationRequest . . . 8
3.6 struct OrchestrationResponse . . . 9
3.7 struct OrchestrationResult . . . 9
3.8 struct OrchestrationServiceRequirement . . . 9
3.9 struct OrchestrationSubscriptionRequest . . . 10
3.10 struct QoSRequirement . . . 10
3.11 struct ServiceInstanceInterfaceResponse . . . 10
3.12 Primitives . . . 11

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `serviceOrchestration` service interface of the SimpleStoreServiceOrchestration system, which provides service orchestration: a consumer asks which provider it should use for a service, and receives the matching, authorized providers with the interfaces to call.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestration` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestration – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestration` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8442 and base path `/serviceorchestration/orchestration`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`pull`	POST	`/pull`
`subscribe`	POST	`/subscribe`
`unsubscribe`	DELETE	`/unsubscribe/{id}`

2 Service Operations

The interfaces of the `serviceOrchestration` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestration` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/orchestration/pull

Operation: pull
Input: OrchestrationRequest
Output: OrchestrationResponse

Returns the store entries of the requester for the requested service definition in priority order; preferred providers first (`ONLY_PREFERRED`: only them); with `MATCHMAKING` only the best entry per service definition.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/pull HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "serviceRequirement": {
    "serviceDefinition": "temperatureReading",
    "operations": [
      "FILTER"
    ],
    "versions": [
      "1.0.0"
    ],
    "alivesAt": "2026-10-22T12:00:00Z",
    "metadataRequirements": [
      {
        "unit": "celsius"
      }
    ],
    "interfaceTemplateNames": [
      "string"
    ],
    "interfaceAddressTypes": [
      "string"
    ],
    "interfacePropertyRequirements": [
      {
        "unit": "celsius"
      }
    ],
    "securityPolicies": [
      "string"
    ],
    "preferredProviders": [
      "string"
    ]
```

```
  },
  "orchestrationFlags": {
    "MATCHMAKING": true
  },
  "qualityRequirements": [
    {
      "type": "PING",
      "operation": "FILTER",
      "requirements": {
        "maxLatency": 100
      }
    }
  ],
  "exclusivityDuration": 60
}
```

Listing 1: An pull invocation.

```
{
  "results": [
    {
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "cloudIdentitifer": "LOCAL",
      "providerName": "TemperatureSensor",
      "serviceDefinitition": "temperatureReading",
      "version": "1.0.0",
      "aliveUntil": "string",
      "exclusiveUntil": "string",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ]
    }
  ],
  "warnings": [
    "string"
  ]
}
```

Listing 2: An pull response.

2.2 POST /serviceorchestration/orchestration/subscribe?trigger=*{trigger}*

Operation: subscribe
Input: OrchestrationSubscriptionRequest
Output: String

Creates (or replaces) a push subscription of the requester for itself; with `trigger=true` the orchestration is executed and pushed immediately. Returns the subscription identifier, 201 for a new subscription.

Query parameters: *trigger* (Boolean, optional, default `false`). List values are repeated or comma-separated.

A successful call returns `201 Created` for a new entry, `200 OK` if an identical entry already existed with the output in the body.

```
POST /serviceorchestration/orchestration/subscribe?trigger=<trigger> HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "targetSystemName": "TemperatureDisplay",
  "orchestrationRequest": {
    "serviceRequirement": {
      "serviceDefinition": "temperatureReading",
      "operations": [
        "FILTER"
      ],
      "versions": [
        "1.0.0"
      ],
      "alivesAt": "2026-10-22T12:00:00Z",
      "metadataRequirements": [
        {
          "unit": "celsius"
        }
      ],
      "interfaceTemplateNames": [
        "string"
      ],
      "interfaceAddressTypes": [
        "string"
      ],
      "interfacePropertyRequirements": [
        {
          "unit": "celsius"
        }
      ],
      "securityPolicies": [
        "string"
      ],
      "preferredProviders": [
        "string"
      ]
    },
    "orchestrationFlags": {
      "MATCHMAKING": true
    },
    "qualityRequirements": [
      {
        "type": "PING",
        "operation": "FILTER",
        "requirements": {
          "maxLatency": 100
        }
      }
    ],
    "exclusivityDuration": 60
  },
  "notifyInterface": {
```

```
    "protocol": "HTTPS",
    "properties": {
      "address": "192.168.0.20",
      "port": "8080",
      "method": "POST",
      "path": "/orchestration/notify"
    }
  },
  "duration": 3600
}
```

Listing 3: An subscribe invocation.

```
"string"
```

Listing 4: An subscribe response.

2.3 DELETE /serviceorchestration/orchestration/unsubscribe/*{id}*

Operation: **unsubscribe**
Input: **String**

Removes a subscription of the requester. Returns 200 if removed, 204 if it did not exist.

Path parameter: *id*.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /serviceorchestration/orchestration/unsubscribe/<id> HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An unsubscribe invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.12, which are used to represent things like hashes, identifiers and texts.

3.1 struct AuthorizationTokenGenerationResponse

A generated token.

Object Field	Value Type	Description
"tokenType"	AuthorizationTokenType	Token type: `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN` or `TRANSLATION_BRIDGE_TOKEN`.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"token"	String	The token (opaque string, Base64 or JSON Web Token).
"usageLimit"	Number	Number of allowed uses (usage-limited tokens).
"expiresAt"	DateTime	UTC expiry of time-limited tokens.

3.2 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.3 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.4 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

3.5 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

3.6 struct OrchestrationResponse

The orchestration result.

Object Field	Value Type	Description
"results"	Array<OrchestrationResult>	The orchestration results, best first.
"warnings"	Array<String>	Warnings of the orchestration, e.g. `autoMatchmaking`, `notExclusive`, `interCloud`.

3.7 struct OrchestrationResult

One provider of the requested service.

Object Field	Value Type	Description
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"cloudIdentitifer"	String	Cloud identifier of the provider (`LOCAL` for the own cloud). The JSON name keeps the upstream spelling.
"providerName"	String	Name of the provider system.
"serviceDefinitition"	String	Service definition name. The JSON name keeps the upstream spelling.
"version"	Version	Version of the service instance.
"aliveUntil"	String	Time until which the service instance is registered (its expiry), if any.
"exclusiveUntil"	String	Time until which the instance is reserved exclusively for the consumer, if reserved.
"metadata"	Object	Metadata of the service instance.
"interfaces"	Array<ServiceInstanceInterface Response>	Interfaces the consumer can use (only matching ones).
"authorizationTokens"	Object<Object<AuthorizationToken GenerationResponse>>	Authorization tokens generated for the consumer, per interface template and operation.

3.8 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

3.9 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

3.10 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

3.11 struct ServiceInstanceInterfaceResponse

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface: `NONE`, `CERT_AUTH` or one of the token policies.
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.12 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
AuthorizationTokenType	One of `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN`, `TRANSLATION_BRIDGE_TOKEN` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestration – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestration MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestration"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestration` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation pull . . . 4
2.2 operation subscribe . . . 5
2.3 operation unsubscribe . . . 6

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct AuthorizationTokenGenerationResponse . . . 7
3.5 struct MetadataRequirement . . . 8
3.6 struct OrchestrationNotifyInterface . . . 8
3.7 struct OrchestrationRequest . . . 8
3.8 struct OrchestrationResponse . . . 8
3.9 struct OrchestrationResult . . . 8
3.10 struct OrchestrationServiceRequirement . . . 9
3.11 struct OrchestrationSubscriptionRequest . . . 9
3.12 struct QoSRequirement . . . 10
3.13 struct ServiceInstanceInterfaceResponse . . . 10
3.14 Primitives . . . 10

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `serviceOrchestration` service interface of the SimpleStoreServiceOrchestration system over MQTT, which provides service orchestration: a consumer asks which provider it should use for a service, and receives the matching, authorized providers with the interfaces to call.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestration` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestration – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/orchestration/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`pull`	`arrowhead/serviceorchestration/orchestration/pull`
`subscribe`	`arrowhead/serviceorchestration/orchestration/subscribe`
`unsubscribe`	`arrowhead/serviceorchestration/orchestration/unsubscribe`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/orchestration/pull`

Operation: pull
Input: OrchestrationRequest
Output: OrchestrationResponse

Returns the store entries of the requester for the requested service definition in priority order; preferred providers first (`ONLY_PREFERRED`: only them); with `MATCHMAKING` only the best entry per service definition.

```
{
  "serviceRequirement": {
    "serviceDefinition": "temperatureReading",
    "operations": [
      "FILTER"
    ],
    "versions": [
      "1.0.0"
    ],
    "alivesAt": "2026-10-22T12:00:00Z",
    "metadataRequirements": [
      {
```

```
        "unit": "celsius"
      }
    ],
    "interfaceTemplateNames": [
      "string"
    ],
    "interfaceAddressTypes": [
      "string"
    ],
    "interfacePropertyRequirements": [
      {
        "unit": "celsius"
      }
    ],
    "securityPolicies": [
      "string"
    ],
    "preferredProviders": [
      "string"
    ]
  },
  "orchestrationFlags": {
    "MATCHMAKING": true
  },
  "qualityRequirements": [
    {
      "type": "PING",
      "operation": "FILTER",
      "requirements": {
        "maxLatency": 100
      }
    }
  ],
  "exclusivityDuration": 60
}
```

Listing 3: Payload of an pull request.

2.2 PUBLISH `arrowhead/serviceorchestration/orchestration/subscribe`

Operation: subscribe
Input: OrchestrationSubscriptionRequest
Output: String

Creates (or replaces) a push subscription of the requester for itself; with `trigger=true` the orchestration is executed and pushed immediately. Returns the subscription identifier, 201 for a new subscription.

Optional entries of `params`: `trigger`.

```
{
  "targetSystemName": "TemperatureDisplay",
  "orchestrationRequest": {
    "serviceRequirement": {
      "serviceDefinition": "temperatureReading",
      "operations": [
        "FILTER"
      ],
      "versions": [
        "1.0.0"
      ],
      "alivesAt": "2026-10-22T12:00:00Z",
      "metadataRequirements": [
        {
          "unit": "celsius"
        }
      ],
      "interfaceTemplateNames": [
```

```
        "string"
      ],
      "interfaceAddressTypes": [
        "string"
      ],
      "interfacePropertyRequirements": [
        {
          "unit": "celsius"
        }
      ],
      "securityPolicies": [
        "string"
      ],
      "preferredProviders": [
        "string"
      ]
    },
    "orchestrationFlags": {
      "MATCHMAKING": true
    },
    "qualityRequirements": [
      {
        "type": "PING",
        "operation": "FILTER",
        "requirements": {
          "maxLatency": 100
        }
      }
    ],
    "exclusivityDuration": 60
  },
  "notifyInterface": {
    "protocol": "HTTPS",
    "properties": {
      "address": "192.168.0.20",
      "port": "8080",
      "method": "POST",
      "path": "/orchestration/notify"
    }
  },
  "duration": 3600
}
```

Listing 4: Payload of an subscribe request.

2.3 PUBLISH arrowhead/serviceorchestration/orchestration/unsubscribe

Operation: unsubscribe
Input: String

Removes a subscription of the requester. Returns 200 if removed, 204 if it did not exist.

```
"3fa85f64-5717-4562-b3fc-2c963f66afa6"
```

Listing 5: Payload of an unsubscribe request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct AuthorizationTokenGenerationResponse

A generated token.

Object Field	Value Type	Description
"tokenType"	AuthorizationTokenType	Token type: `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN` or `TRANSLATION_BRIDGE_TOKEN`.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"token"	String	The token (opaque string, Base64 or JSON Web Token).
"usageLimit"	Number	Number of allowed uses (usage-limited tokens).
"expiresAt"	DateTime	UTC expiry of time-limited tokens.

3.5 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.6 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

3.7 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

3.8 struct OrchestrationResponse

The orchestration result.

Object Field	Value Type	Description
"results"	Array<OrchestrationResult>	The orchestration results, best first.
"warnings"	Array<String>	Warnings of the orchestration, e.g. `autoMatchmaking`, `notExclusive`, `interCloud`.

3.9 struct OrchestrationResult

One provider of the requested service.

Object Field	Value Type	Description
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"cloudIdentitifer"	String	Cloud identifier of the provider (`LOCAL` for the own cloud). The JSON name keeps the upstream spelling.
"providerName"	String	Name of the provider system.
"serviceDefinitition"	String	Service definition name. The JSON name keeps the upstream spelling.
"version"	Version	Version of the service instance.
"aliveUntil"	String	Time until which the service instance is registered (its expiry), if any.
"exclusiveUntil"	String	Time until which the instance is reserved exclusively for the consumer, if reserved.
"metadata"	Object	Metadata of the service instance.
"interfaces"	Array<ServiceInstanceInterface Response>	Interfaces the consumer can use (only matching ones).
"authorizationTokens"	Object<Object<AuthorizationToken GenerationResponse>>	Authorization tokens generated for the consumer, per interface template and operation.

3.10 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

3.11 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

3.12 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

3.13 struct ServiceInstanceInterfaceResponse

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface: `NONE`, `CERT_AUTH` or one of the token policies.
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.14 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
AuthorizationTokenType	One of `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN`, `TRANSLATION_BRIDGE_TOKEN` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestration – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

[6] ——, “serviceOrchestration HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationHistoryManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationHistoryManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationHistoryManagement` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct OrchestrationHistoryQueryRequest . . . 6
3.3 struct OrchestrationHistoryResponse . . . 6
3.4 struct OrchestrationJob . . . 6
3.5 struct Page . . . 7
3.6 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `serviceOrchestrationHistoryManagement` service interface of the SimpleStoreServiceOrchestration system, which provides query of the orchestration history: the pull and push orchestration jobs, their status and outcome.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationHistoryManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationHistoryManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestrationHistoryManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8442 and base path `/serviceorchestration/orchestration/mgmt/history`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query`	POST	`/query`

2 Service Operations

The interfaces of the `serviceOrchestrationHistoryManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestrationHistoryManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/orchestration/mgmt/history/query

Operation: query
Input: OrchestrationHistoryQueryRequest
Output: OrchestrationHistoryResponse

Returns a page of orchestration jobs matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/history/query HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ids": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ],
  "statuses": [
    "string"
  ],
  "type": "IPV4",
  "requesterSystems": [
    "TemperatureDisplay"
  ],
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ],
  "subscriptionIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Listing 1: An query invocation.

```
{
```

```
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "status": "DONE",
      "type": "IPV4",
      "requesterSystem": "TemperatureDisplay",
      "targetSystem": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "subscriptionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "message": "1 local result: TemperatureSensor",
      "createdAt": "2026-10-22T12:00:00Z",
      "startedAt": "2026-10-22T12:00:00Z",
      "finishedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```


Listing 2: An query response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.6, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct OrchestrationHistoryQueryRequest

Paged filter for orchestration jobs.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ids"	Array<String>	Job identifiers (UUID) to match.
"statuses"	Array<String>	Job statuses to match.
"type"	String	Orchestration type to match: `PULL` or `PUSH`.
"requesterSystems"	Array<String>	Requester system names to match.
"targetSystems"	Array<String>	Target system names.
"serviceDefinitions"	Array<String>	Service definition names to match.
"subscriptionIds"	Array<String>	Subscription identifiers.

3.3 struct OrchestrationHistoryResponse

List of orchestration jobs.

Object Field	Value Type	Description
"entries"	Array<OrchestrationJob>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.4 struct OrchestrationJob

One orchestration job (pull or push).

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the orchestration job.
"status"	String	Status of the job: `PENDING`, `IN_PROGRESS`, `DONE` or `ERROR`.
"type"	String	Orchestration type: `PULL` or `PUSH`.
"requesterSystem"	String	Name of the system that requested the orchestration.
"targetSystem"	String	Name of the consumer system the orchestration was made for.
"serviceDefinition"	String	Service definition name.
"subscriptionId"	String	Subscription that started the push job.
"message"	String	Status message, e.g. `1 local result: DataManager`.
"createdAt"	DateTime	UTC time of creation.
"startedAt"	DateTime	UTC time when the job started.
"finishedAt"	DateTime	UTC time when the job finished.

3.5 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationHistoryManagement – Service Description (SimpleStoreService-Orchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationHistoryManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationHistoryManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationHistoryManagement` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation query . . . 4

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct OrchestrationHistoryQueryRequest . . . 6
3.5 struct OrchestrationHistoryResponse . . . 7
3.6 struct OrchestrationJob . . . 7
3.7 struct Page . . . 7
3.8 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `serviceOrchestrationHistoryManagement` service interface of the SimpleStoreServiceOrchestration system over MQTT, which provides query of the orchestration history: the pull and push orchestration jobs, their status and outcome.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationHistoryManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationHistoryManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/orchestration/management/history/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`query`	`arrowhead/serviceorchestration/orchestration/management/history/query`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/orchestration/management/history/query`

Operation: query
Input: OrchestrationHistoryQueryRequest
Output: OrchestrationHistoryResponse

Returns a page of orchestration jobs matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ids": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ],
  "statuses": [
    "string"
  ],
  "type": "IPV4",
  "requesterSystems": [
```

```
    "TemperatureDisplay"
  ],
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ],
  "subscriptionIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Listing 3: Payload of an query request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct OrchestrationHistoryQueryRequest

Paged filter for orchestration jobs.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ids"	Array<String>	Job identifiers (UUID) to match.
"statuses"	Array<String>	Job statuses to match.
"type"	String	Orchestration type to match: `PULL` or `PUSH`.
"requesterSystems"	Array<String>	Requester system names to match.
"targetSystems"	Array<String>	Target system names.
"serviceDefinitions"	Array<String>	Service definition names to match.
"subscriptionIds"	Array<String>	Subscription identifiers.

3.5 struct OrchestrationHistoryResponse

List of orchestration jobs.

Object Field	Value Type	Description
"entries"	Array<OrchestrationJob>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.6 struct OrchestrationJob

One orchestration job (pull or push).

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the orchestration job.
"status"	String	Status of the job: `PENDING`, `IN_PROGRESS`, `DONE` or `ERROR`.
"type"	String	Orchestration type: `PULL` or `PUSH`.
"requesterSystem"	String	Name of the system that requested the orchestration.
"targetSystem"	String	Name of the consumer system the orchestration was made for.
"serviceDefinition"	String	Service definition name.
"subscriptionId"	String	Subscription that started the push job.
"message"	String	Status message, e.g. `1 local result: DataManager`.
"createdAt"	DateTime	UTC time of creation.
"startedAt"	DateTime	UTC time when the job started.
"finishedAt"	DateTime	UTC time when the job finished.

3.7 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.8 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationHistoryManagement – Service Description (SimpleStoreService-Orchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-SD.pdf.

[6] ——, “serviceOrchestrationHistoryManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationPushManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationPushManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationPushManagement` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation subscribe . . . 4
2.2 operation unsubscribe . . . 6
2.3 operation trigger . . . 6
2.4 operation query . . . 7

3 Data Models **9**
3.1 struct ErrorMessage . . . 9
3.2 struct MetadataRequirement . . . 9
3.3 struct OrchestrationJob . . . 9
3.4 struct OrchestrationNotifyInterface . . . 9
3.5 struct OrchestrationPushJobListResponse . . . 10
3.6 struct OrchestrationPushTrigger . . . 10
3.7 struct OrchestrationRequest . . . 10
3.8 struct OrchestrationServiceRequirement . . . 10
3.9 struct OrchestrationSubscriptionListRequest . . . 11
3.10 struct OrchestrationSubscriptionListResponse . . . 11
3.11 struct OrchestrationSubscriptionQueryRequest . . . 11
3.12 struct OrchestrationSubscriptionRequest . . . 11
3.13 struct OrchestrationSubscriptionResponse . . . 12
3.14 struct Page . . . 12
3.15 struct QoSRequirement . . . 12
3.16 Primitives . . . 13

4 References **14**

5 Revision History **15**
5.1 Amendments . . . 15
5.2 Quality Assurance . . . 15

1 Overview

This document describes the `serviceOrchestrationPushManagement` service interface of the SimpleStoreServiceOrchestration system, which provides operator management of push orchestration: subscriptions made on behalf of consumers, and triggering of push orchestration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationPushManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationPushManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestrationPushManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8442 and base path `/serviceorchestration/orchestration/mgmt/push`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`subscribe`	POST	`/subscribe`
`unsubscribe`	DELETE	`/unsubscribe`
`trigger`	POST	`/trigger`
`query`	POST	`/query`

2 Service Operations

The interfaces of the `serviceOrchestrationPushManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestrationPushManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/orchestration/mgmt/push/subscribe

Operation: subscribe
Input: OrchestrationSubscriptionListRequest
Output: OrchestrationSubscriptionListResponse

Creates subscriptions on behalf of target systems.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/push/subscribe HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "subscriptions": [
    {
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
          "operations": [
            "FILTER"
          ],
          "versions": [
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
```

```
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "duration": 3600
    }
  ]
}
```

Listing 1: An subscribe invocation.

```
{
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ownerSystemName": "TemperatureDisplay",
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
          "operations": [
            "FILTER"
          ],
          "versions": [
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
```

```
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "expiredAt": "2026-10-22T12:00:00Z",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An subscribe response.

2.2 DELETE /serviceorchestration/orchestration/mgmt/push/unsubscribe?ids={ids}

Operation: unsubscribe
Input: Array<String>

Removes subscriptions by identifier.
Query parameters: *ids* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /serviceorchestration/orchestration/mgmt/push/unsubscribe?ids=<ids> HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An unsubscribe invocation.

2.3 POST /serviceorchestration/orchestration/mgmt/push/trigger

Operation: trigger
Input: OrchestrationPushTrigger
Output: OrchestrationPushJobListResponse

Starts push orchestration for the selected subscriptions (by target system or identifier); returns the started jobs.
A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/push/trigger HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "subscriptionIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Listing 4: An trigger invocation.

```
{
  "jobs": [
    {
```

```
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "status": "DONE",
      "type": "IPV4",
      "requesterSystem": "TemperatureDisplay",
      "targetSystem": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "subscriptionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "message": "1 local result: TemperatureSensor",
      "createdAt": "2026-10-22T12:00:00Z",
      "startedAt": "2026-10-22T12:00:00Z",
      "finishedAt": "2026-10-22T12:00:00Z"
    }
  ]
}
```

Listing 5: An trigger response.

2.4 POST /serviceorchestration/orchestration/mgmt/push/query

Operation: query
Input: OrchestrationSubscriptionQueryRequest
Output: OrchestrationSubscriptionListResponse

Returns a page of subscriptions matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/push/query HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ownerSystems": [
    "string"
  ],
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ]
}
```

Listing 6: An query invocation.

```
{
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ownerSystemName": "TemperatureDisplay",
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
          "operations": [
            "FILTER"
          ],
          "versions": [
```

```
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "expiredAt": "2026-10-22T12:00:00Z",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```


Listing 7: An query response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.16, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.3 struct OrchestrationJob

One orchestration job (pull or push).

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the orchestration job.
"status"	String	Status of the job: `PENDING`, `IN_PROGRESS`, `DONE` or `ERROR`.
"type"	String	Orchestration type: `PULL` or `PUSH`.
"requesterSystem"	String	Name of the system that requested the orchestration.
"targetSystem"	String	Name of the consumer system the orchestration was made for.
"serviceDefinition"	String	Service definition name.
"subscriptionId"	String	Subscription that started the push job.
"message"	String	Status message, e.g. `1 local result: DataManager`.
"createdAt"	DateTime	UTC time of creation.
"startedAt"	DateTime	UTC time when the job started.
"finishedAt"	DateTime	UTC time when the job finished.

3.4 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

3.5 struct OrchestrationPushJobListResponse

Push jobs started by a trigger.

Object Field	Value Type	Description
"jobs"	Array<OrchestrationJob>	Orchestration jobs started by the trigger.

3.6 struct OrchestrationPushTrigger

Selects the subscriptions to execute now.

Object Field	Value Type	Description
"targetSystems"	Array<String>	Consumers whose subscriptions are executed.
"subscriptionIds"	Array<String>	Subscriptions to execute.

3.7 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

3.8 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

3.9 struct OrchestrationSubscriptionListRequest

Subscriptions to create on behalf of consumers.

Object Field	Value Type	Description
"subscriptions"	Array<OrchestrationSubscription Request>	Subscriptions to create.

3.10 struct OrchestrationSubscriptionListResponse

List of subscriptions.

Object Field	Value Type	Description
"entries"	Array<OrchestrationSubscription Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.11 struct OrchestrationSubscriptionQueryRequest

Paged filter for subscriptions.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ownerSystems"	Array<String>	Subscription owner names to match.
"targetSystems"	Array<String>	Target system names.
"serviceDefinitions"	Array<String>	Service definition names to match.

3.12 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

3.13 struct OrchestrationSubscriptionResponse

A stored subscription.

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the subscription.
"ownerSystemName"	String	Name of the system that created the subscription.
"targetSystemName"	String	Name of the consumer system the push orchestration is made for.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"expiredAt"	DateTime	UTC time when the subscription expires.
"createdAt"	DateTime	UTC time of creation.

3.14 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.15 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

3.16 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationPushManagement – Service Description (SimpleStoreService-Orchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationPushManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationPushManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationPushManagement` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation subscribe . . . 4
2.2 operation trigger . . . 5
2.3 operation unsubscribe . . . 6
2.4 operation query . . . 6

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct MetadataRequirement . . . 7
3.5 struct OrchestrationJob . . . 7
3.6 struct OrchestrationNotifyInterface . . . 8
3.7 struct OrchestrationPushJobListResponse . . . 8
3.8 struct OrchestrationPushTrigger . . . 8
3.9 struct OrchestrationRequest . . . 8
3.10 struct OrchestrationServiceRequirement . . . 9
3.11 struct OrchestrationSubscriptionListRequest . . . 9
3.12 struct OrchestrationSubscriptionListResponse . . . 9
3.13 struct OrchestrationSubscriptionQueryRequest . . . 9
3.14 struct OrchestrationSubscriptionRequest . . . 10
3.15 struct OrchestrationSubscriptionResponse . . . 10
3.16 struct Page . . . 10
3.17 struct QoSRequirement . . . 10
3.18 Primitives . . . 11

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `serviceOrchestrationPushManagement` service interface of the SimpleStoreServiceOrchestration system over MQTT, which provides operator management of push orchestration: subscriptions made on behalf of consumers, and triggering of push orchestration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationPushManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationPushManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/orchestration/management/push/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`subscribe`	`arrowhead/serviceorchestration/orchestration/management/push/subscribe`
`trigger`	`arrowhead/serviceorchestration/orchestration/management/push/trigger`
`unsubscribe`	`arrowhead/serviceorchestration/orchestration/management/push/unsubscribe`
`query`	`arrowhead/serviceorchestration/orchestration/management/push/query`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/orchestration/management/push/subscribe`

Operation: subscribe
Input: OrchestrationSubscriptionListRequest
Output: OrchestrationSubscriptionListResponse

Creates subscriptions on behalf of target systems.

```
{
  "subscriptions": [
    {
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
```

```
          "operations": [
            "FILTER"
          ],
          "versions": [
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "duration": 3600
    }
  ]
}
```

Listing 3: Payload of an subscribe request.

2.2 PUBLISH arrowhead/serviceorchestration/orchestration/management/push/trigger

Operation: trigger
Input: OrchestrationPushTrigger
Output: OrchestrationPushJobListResponse

Starts push orchestration for the selected subscriptions (by target system or identifier); returns the started jobs.

```
{
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "subscriptionIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
```

```
}
```

Listing 4: Payload of an trigger request.

2.3 PUBLISH arrowhead/serviceorchestration/orchestration/management/push/unsubscribe

Operation: unsubscribe
Input: Array<String>

Removes subscriptions by identifier.

```
[
  "3fa85f64-5717-4562-b3fc-2c963f66afa6"
]
```

Listing 5: Payload of an unsubscribe request.

2.4 PUBLISH arrowhead/serviceorchestration/orchestration/management/push/query

Operation: query
Input: OrchestrationSubscriptionQueryRequest
Output: OrchestrationSubscriptionListResponse

Returns a page of subscriptions matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ownerSystems": [
    "string"
  ],
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ]
}
```

Listing 6: Payload of an query request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.5 struct OrchestrationJob

One orchestration job (pull or push).

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the orchestration job.
"status"	String	Status of the job: `PENDING`, `IN_PROGRESS`, `DONE` or `ERROR`.
"type"	String	Orchestration type: `PULL` or `PUSH`.
"requesterSystem"	String	Name of the system that requested the orchestration.
"targetSystem"	String	Name of the consumer system the orchestration was made for.
"serviceDefinition"	String	Service definition name.
"subscriptionId"	String	Subscription that started the push job.
"message"	String	Status message, e.g. `1 local result: DataManager`.
"createdAt"	DateTime	UTC time of creation.
"startedAt"	DateTime	UTC time when the job started.
"finishedAt"	DateTime	UTC time when the job finished.

3.6 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

3.7 struct OrchestrationPushJobListResponse

Push jobs started by a trigger.

Object Field	Value Type	Description
"jobs"	Array<OrchestrationJob>	Orchestration jobs started by the trigger.

3.8 struct OrchestrationPushTrigger

Selects the subscriptions to execute now.

Object Field	Value Type	Description
"targetSystems"	Array<String>	Consumers whose subscriptions are executed.
"subscriptionIds"	Array<String>	Subscriptions to execute.

3.9 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

3.10 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

3.11 struct OrchestrationSubscriptionListRequest

Subscriptions to create on behalf of consumers.

Object Field	Value Type	Description
"subscriptions"	Array<OrchestrationSubscription Request>	Subscriptions to create.

3.12 struct OrchestrationSubscriptionListResponse

List of subscriptions.

Object Field	Value Type	Description
"entries"	Array<OrchestrationSubscription Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.13 struct OrchestrationSubscriptionQueryRequest

Paged filter for subscriptions.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ownerSystems"	Array<String>	Subscription owner names to match.
"targetSystems"	Array<String>	Target system names.
"serviceDefinitions"	Array<String>	Service definition names to match.

3.14 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

3.15 struct OrchestrationSubscriptionResponse

A stored subscription.

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the subscription.
"ownerSystemName"	String	Name of the system that created the subscription.
"targetSystemName"	String	Name of the consumer system the push orchestration is made for.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"expiredAt"	DateTime	UTC time when the subscription expires.
"createdAt"	DateTime	UTC time of creation.

3.16 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.17 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

3.18 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationPushManagement – Service Description (SimpleStoreService-Orchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-SD.pdf.

[6] ——, “serviceOrchestrationPushManagement HTTP/TLS/JSON – Interface Design Description (Simple-StoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationSimpleStoreManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationSimpleStoreManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationSimpleStoreManagement` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query . . . 4
2.2 operation remove . . . 5
2.3 operation create . . . 5
2.4 operation modify-priorities . . . 6

3 Data Models **7**
3.1 struct ErrorMessage . . . 7
3.2 struct OrchestrationSimpleStoreListRequest . . . 7
3.3 struct OrchestrationSimpleStoreListResponse . . . 7
3.4 struct OrchestrationSimpleStoreQueryRequest . . . 7
3.5 struct OrchestrationSimpleStoreRequest . . . 7
3.6 struct OrchestrationSimpleStoreResponse . . . 8
3.7 struct Page . . . 8
3.8 struct PriorityRequest . . . 8
3.9 Primitives . . . 8

4 References **10**

5 Revision History **11**
5.1 Amendments . . . 11
5.2 Quality Assurance . . . 11

1 Overview

This document describes the `serviceOrchestrationSimpleStoreManagement` service interface of the SimpleStoreServiceOrchestration system, which provides operator management of the orchestration store: the static rules telling which service instance a consumer shall use, with priorities.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationSimpleStoreManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationSimpleStoreManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestrationSimpleStoreManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8442 and base path `/serviceorchestration/orchestration/mgmt/simple-store`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query`	POST	`/query`
`remove`	DELETE	`/remove`
`create`	POST	`/create`
`modify-priorities`	PUT	`/modify-priorities`

2 Service Operations

The interfaces of the `serviceOrchestrationSimpleStoreManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestrationSimpleStoreManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/orchestration/mgmt/simple-store/query

Operation: query
Input: OrchestrationSimpleStoreQueryRequest
Output: OrchestrationSimpleStoreListResponse

Returns a page of store entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/simple-store/query HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ids": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ],
  "consumerNames": [
    "string"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ],
  "serviceInstanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "minPriority": 1,
  "maxPriority": 10,
  "createdBy": "Sysop"
}
```

Listing 1: An query invocation.

```
{
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "consumer": "TemperatureDisplay",
```

```
      "serviceDefinition": "temperatureReading",
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "priority": 1,
      "createdBy": "Sysop",
      "updatedBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An query response.

2.2 DELETE /serviceorchestration/orchestration/mgmt/simple-store/remove?uuids=*{uuids}*

Operation: remove
Input: Array<String>

Removes store entries by identifier.
Query parameters: *uuids* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /serviceorchestration/orchestration/mgmt/simple-store/remove?uuids=<uuids> HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An remove invocation.

2.3 POST /serviceorchestration/orchestration/mgmt/simple-store/create

Operation: create
Input: OrchestrationSimpleStoreListRequest
Output: OrchestrationSimpleStoreListResponse

Creates store entries; the service definition is taken from the instance identifier.
A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/simple-store/create HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "candidates": [
    {
      "consumer": "TemperatureDisplay",
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "priority": 1
    }
  ]
}
```

Listing 4: An create invocation.

```
{
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
```

```
      "consumer": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "priority": 1,
      "createdBy": "Sysop",
      "updatedBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 5: An create response.

2.4 PUT /serviceorchestration/orchestration/mgmt/simple-store/modify-priorities

Operation: modify-priorities
Input: PriorityRequest
Output: OrchestrationSimpleStoreListResponse

Changes the priorities of store entries.

A successful call returns `200 OK` with the output in the body.

```
PUT /serviceorchestration/orchestration/mgmt/simple-store/modify-priorities HTTP/1.1
Host: serviceorchestration-simple:8442
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "3fa85f64-5717-4562-b3fc-2c963f66afa6": 2
}
```

Listing 6: An modify-priorities invocation.

```
{
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "consumer": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "priority": 1,
      "createdBy": "Sysop",
      "updatedBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 7: An modify-priorities response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.9, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct OrchestrationSimpleStoreListRequest

Store entries to create.

Object Field	Value Type	Description
"candidates"	Array<OrchestrationSimpleStore Request>	Store entries to create.

3.3 struct OrchestrationSimpleStoreListResponse

List of store entries.

Object Field	Value Type	Description
"entries"	Array<OrchestrationSimpleStore Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.4 struct OrchestrationSimpleStoreQueryRequest

Paged filter for store entries.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ids"	Array<String>	Store entry identifiers (UUID) to match.
"consumerNames"	Array<String>	Consumer system names to match.
"serviceDefinitions"	Array<String>	Service definition names to match.
"serviceInstanceIds"	Array<String>	Service instance identifiers to match.
"minPriority"	Number	Lower bound of the priority.
"maxPriority"	Number	Upper bound of the priority.
"createdBy"	String	Creator system to match.

3.5 struct OrchestrationSimpleStoreRequest

One orchestration rule: consumer, service instance and priority.

Object Field	Value Type	Description
"consumer"	String	Consumer system the rule applies to.
"serviceInstanceId"	String	Service instance offered to the consumer.
"priority"	Number	Priority of the rule; 1 is the highest.

3.6 struct OrchestrationSimpleStoreResponse

A stored orchestration rule.

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the store entry.
"consumer"	String	Name of the consumer system.
"serviceDefinition"	String	Service definition name.
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"priority"	Number	Priority of the entry; 1 is the highest.
"createdBy"	String	Name of the system that created the record.
"updatedBy"	String	Name of the system that changed the record last.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.7 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.8 struct PriorityRequest

A JSON object mapping store entry identifiers (UUID) to new priorities, e.g. `{"123e4567-e89b-12d3-a456-426614174000": 5}`.

3.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationSimpleStoreManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationSimpleStoreManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationSimpleStoreManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationSimpleStoreManagement` service of the SimpleStoreServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation create . . . 4
2.2 operation query . . . 5
2.3 operation remove . . . 5
2.4 operation modify-priorities . . . 5

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct OrchestrationSimpleStoreListRequest . . . 7
3.5 struct OrchestrationSimpleStoreListResponse . . . 7
3.6 struct OrchestrationSimpleStoreQueryRequest . . . 8
3.7 struct OrchestrationSimpleStoreRequest . . . 8
3.8 struct OrchestrationSimpleStoreResponse . . . 8
3.9 struct Page . . . 8
3.10 struct PriorityRequest . . . 9
3.11 Primitives . . . 9

4 References **10**

5 Revision History **11**
5.1 Amendments . . . 11
5.2 Quality Assurance . . . 11

1 Overview

This document describes the `serviceOrchestrationSimpleStoreManagement` service interface of the SimpleStoreServiceOrchestration system over MQTT, which provides operator management of the orchestration store: the static rules telling which service instance a consumer shall use, with priorities.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationSimpleStoreManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationSimpleStoreManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/orchestration/management/store/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`create`	`arrowhead/serviceorchestration/orchestration/management/store/create`
`query`	`arrowhead/serviceorchestration/orchestration/management/store/query`
`remove`	`arrowhead/serviceorchestration/orchestration/management/store/remove`
`modify-priorities`	`arrowhead/serviceorchestration/orchestration/management/store/modify-priorities`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/orchestration/management/store/create`

Operation: create
Input: OrchestrationSimpleStoreListRequest
Output: OrchestrationSimpleStoreListResponse

Creates store entries; the service definition is taken from the instance identifier.

```
{
  "candidates": [
    {
      "consumer": "TemperatureDisplay",
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "priority": 1
    }
```

```
  ]
}
```

Listing 3: Payload of an create request.

2.2 PUBLISH arrowhead/serviceorchestration/orchestration/management/store/query

Operation: query
Input: OrchestrationSimpleStoreQueryRequest
Output: OrchestrationSimpleStoreListResponse

Returns a page of store entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ids": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ],
  "consumerNames": [
    "string"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ],
  "serviceInstanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "minPriority": 1,
  "maxPriority": 10,
  "createdBy": "Sysop"
}
```

Listing 4: Payload of an query request.

2.3 PUBLISH arrowhead/serviceorchestration/orchestration/management/store/remove

Operation: remove
Input: Array<String>

Removes store entries by identifier.

```
[
  "string"
]
```

Listing 5: Payload of an remove request.

2.4 PUBLISH `arrowhead/serviceorchestration/orchestration/management/store/modify-priorities`

Operation: modify-priorities
Input: PriorityRequest
Output: OrchestrationSimpleStoreListResponse

Changes the priorities of store entries.

```
{
  "3fa85f64-5717-4562-b3fc-2c963f66afa6": 2
}
```

Listing 6: Payload of an modify-priorities request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct OrchestrationSimpleStoreListRequest

Store entries to create.

Object Field	Value Type	Description
"candidates"	Array<OrchestrationSimpleStoreRequest>	Store entries to create.

3.5 struct OrchestrationSimpleStoreListResponse

List of store entries.

Object Field	Value Type	Description
"entries"	Array<OrchestrationSimpleStore Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.6 struct OrchestrationSimpleStoreQueryRequest

Paged filter for store entries.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ids"	Array<String>	Store entry identifiers (UUID) to match.
"consumerNames"	Array<String>	Consumer system names to match.
"serviceDefinitions"	Array<String>	Service definition names to match.
"serviceInstanceIds"	Array<String>	Service instance identifiers to match.
"minPriority"	Number	Lower bound of the priority.
"maxPriority"	Number	Upper bound of the priority.
"createdBy"	String	Creator system to match.

3.7 struct OrchestrationSimpleStoreRequest

One orchestration rule: consumer, service instance and priority.

Object Field	Value Type	Description
"consumer"	String	Consumer system the rule applies to.
"serviceInstanceId"	String	Service instance offered to the consumer.
"priority"	Number	Priority of the rule; 1 is the highest.

3.8 struct OrchestrationSimpleStoreResponse

A stored orchestration rule.

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the store entry.
"consumer"	String	Name of the consumer system.
"serviceDefinition"	String	Service definition name.
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"priority"	Number	Priority of the entry; 1 is the highest.
"createdBy"	String	Name of the system that created the record.
"updatedBy"	String	Name of the system that changed the record last.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.9 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.10 struct PriorityRequest

A JSON object mapping store entry identifiers (UUID) to new priorities, e.g. `{"123e4567-e89b-12d3-a456-426614174000": 5}`.

3.11 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationSimpleStoreManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-SD.pdf.

[6] ——, “serviceOrchestrationSimpleStoreManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the SimpleStoreServiceOrchestration system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration 3

2 Service Operations **4**
2.1 operation echo 4
2.2 operation power 4
2.3 operation status 4

3 Data Models **6**
3.1 struct Observation 6
3.2 Primitives 6

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of SimpleStoreServiceOrchestration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, SimpleStoreServiceOrchestration registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.SimpleStoreServiceOrchestration.
    SimpleStoreServiceOrchestration",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
```

```
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher connects to the broker (`sysmonitor.broker.address`, default `amqp-broker`; port `sysmonitor.broker.port`, default 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. The local cloud name is taken from `sysmonitor.local.cloud` or the system certificate. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.SimpleStoreServiceOrchestration.SimpleStoreServiceOrchestration.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.SimpleStoreServiceOrchestration.SimpleStoreServiceOrchestration.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.SimpleStoreServiceOrchestration.SimpleStoreServiceOrchestration.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "SimpleStoreServiceOrchestration-echo-1792670400000-42",
  "providerSystem": "SimpleStoreServiceOrchestration",
  "serviceInstance": "SimpleStoreServiceOrchestration",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "SimpleStoreServiceOrchestration is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
  }
```

```
18 }
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power` or `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Interface overview

Mandatory mutual TLS, Arrowhead token/certificate binding, JSON request and response documents. Echo responses, orchestration subscription IDs, authorization public keys and encryption IV responses are text. Certificate-only operations and management are distinguished in the table.

1.1 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/serviceorchestration/general/mgmt/get-config	management
POST	/serviceorchestration/general/mgmt/logs	management
POST	/serviceorchestration/orchestration/pull	identity
POST	/serviceorchestration/orchestration/subscribe	identity
DELETE	/serviceorchestration/orchestration/unsubscribe/{id}	identity
POST	/serviceorchestration/orchestration/mgmt/push/subscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/trigger	management
DELETE	/serviceorchestration/orchestration/mgmt/push/unsubscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/query	management
POST	/serviceorchestration/orchestration/mgmt/history/query	management
POST	/serviceorchestration/orchestration/mgmt/simple-store/create	management
POST	/serviceorchestration/orchestration/mgmt/simple-store/query	management
PUT	/serviceorchestration/orchestration/mgmt/simple-store/modify-priorities	management
DELETE	/serviceorchestration/orchestration/mgmt/simple-store/remove	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

2 Limitations

Stored candidate priority selection, preferred providers, matchmaking, history, subscriptions and explicit HTTPS push jobs are implemented. Store results intentionally retain Java's sparse result shape. Full Java query/priority-normalization semantics, event-driven reevaluation, MQTT callbacks and Java state import are not ported. Unsupported simple-store criteria are reported as ignored warnings.